

A black snake is coiled on the keyboard of a laptop. The snake's head is raised, and its tongue is flicking out. The laptop is dark, and the background is black.

S4.1 · Python: Entorno y Lenguaje

Bienvenido a la sesión 4.1 del curso. Hoy sentamos las bases de Python, el lenguaje favorito del mundo de la Inteligencia Artificial. Aprenderemos a configurar nuestro entorno de trabajo, dominar la sintaxis esencial y escribir nuestros primeros programas.

NIVEL INICIAL · INTERMEDIO

PYTHON PARA IA

Agenda de hoy

Estos son los bloques de contenido que recorreremos a lo largo de la sesión:

01

¿Por qué Python en IA?

Razones por las que Python domina el ecosistema de la IA.

03

Variables, tipos y operadores

Los bloques básicos del lenguaje.

05

Entrada / Salida

Cómo tu programa se comunica con el usuario.

02

Entorno de trabajo

Python, entornos virtuales, pip, Jupyter, VS Code y Git.

04

Control de flujo

Condicionales y bucles para tomar decisiones y repetir acciones.

06

Python vs Java y ejercicios

Comparativa práctica y puesta en práctica.

¿Por qué Python en IA?

Python se ha convertido en el lenguaje **de facto** de la Inteligencia Artificial. No es casualidad: su diseño prioriza la legibilidad y la productividad, lo que permite a investigadores y desarrolladores centrarse en los problemas, no en la sintaxis.



Las razones clave de su éxito



Sintaxis legible

Su código se parece al pseudocódigo. Aprender es más rápido y los errores son más fáciles de detectar.



Gran comunidad

Millones de desarrolladores, tutoriales, foros y paquetes listos para usar. Nunca estarás solo ante un problema.



Ecosistema IA

Bibliotecas como NumPy, Pandas, TensorFlow y PyTorch hacen que Python sea insustituible en ciencia de datos e IA.

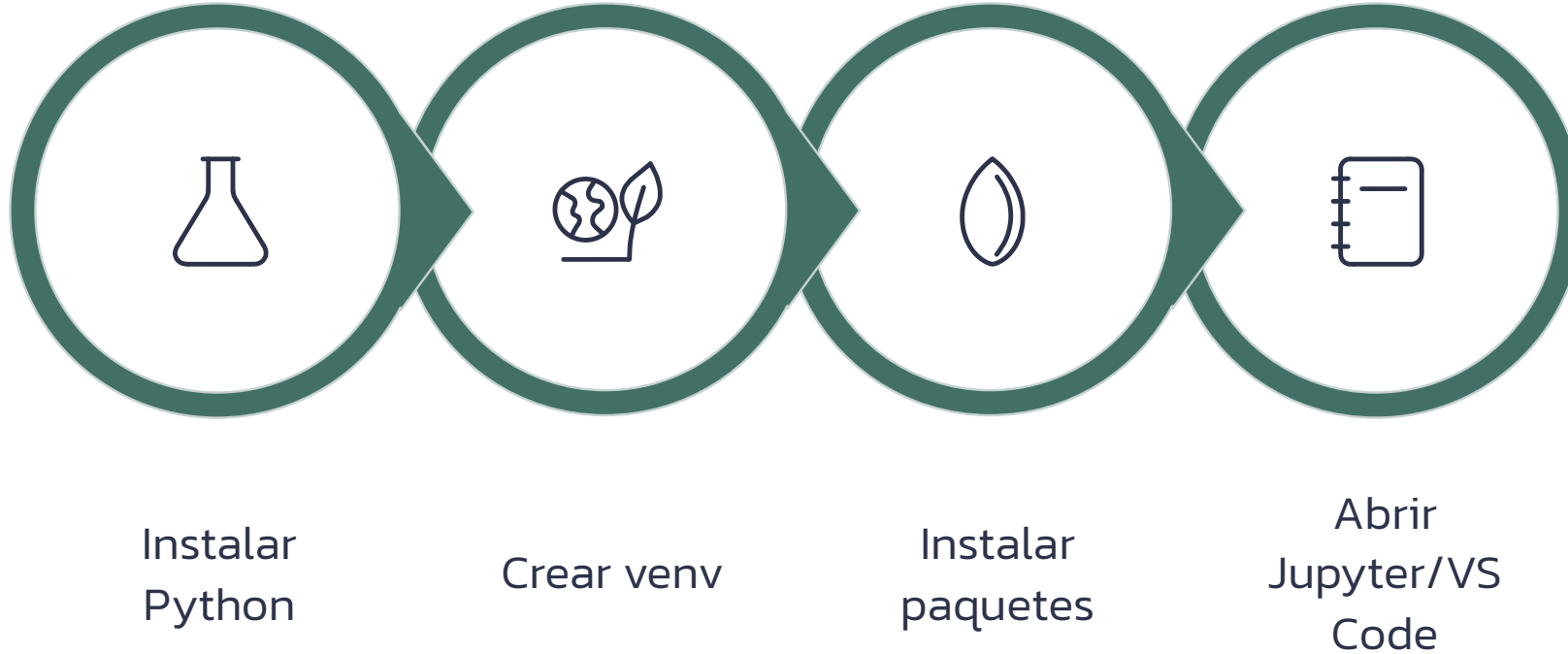


Prototipado rápido

Pasar de una idea a un experimento funcional toma minutos, no horas. Ideal para iterar en proyectos de IA.

Configurando tu entorno Python

Un entorno bien configurado es la base de cualquier proyecto. Antes de escribir una sola línea de código IA, necesitamos instalar y entender las herramientas que usaremos a diario.



Las herramientas del ecosistema



Python

El intérprete del lenguaje. Descárgalo desde **python.org** (versión 3.10 o superior). Incluye la librería estándar y el gestor de paquetes `pip`.



Entornos virtuales

Un entorno virtual (`venv`) aísla las dependencias de cada proyecto. Así evitas conflictos entre versiones de paquetes. Créalo con `python -m venv .venv` y actívalo con `source .venv/bin/activate`.



pip

El gestor de paquetes de Python. Instala bibliotecas con `pip install nombre_paquete`. Guarda las dependencias del proyecto con `pip freeze > requirements.txt`.



Jupyter Notebook

Entorno interactivo que combina código, texto y gráficos en un mismo documento. Perfecto para exploración de datos y experimentos de IA. Instálalo con `pip install notebook`.



VS Code

Editor de código ligero y muy potente. Con la extensión de Python ofrece autocompletado, depuración y soporte para Jupyter integrado.

Git básico: tu historial de cambios

Git es el sistema de control de versiones más utilizado del mundo. Te permite guardar «fotografías» de tu código a lo largo del tiempo, colaborar en equipo y recuperar versiones anteriores si algo sale mal.

1

git init

Inicializa un repositorio en tu carpeta de proyecto.

2

git add

Marca los ficheros modificados que quieres guardar.

3

git commit

Guarda la «fotografía» con un mensaje descriptivo.

4

git push

Sube los cambios al repositorio remoto (p. ej. GitHub).

```
user@devbox:~/projects/webapp$ git init
Initialized empty Git repository in /home/user/projects/webapp/.git/
On branch main
Your branch is up to date with 'origin/main'.
Changes not staged for commit:
  modified:   src/components/Header.tsx
  modified:   src/styles/main.css
no changes added to commit (use "git add" and/or "git commit -a")

user@devbox:~/projects/webapp$ git add .
user@devbox:~/projects/webapp$ git commit -m "fix: update header styling"
[main abc1234] fix: update header styling
2 files changed, 14 insertions(+), 3 deletions(-)
user@devbox:~/projects/webapp$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads.
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 412 bytes | 412.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:user/webapp@git
 f8e2c1d..abc1234  main -> main
user@devbox:~/projects/webapp$
```

Variables: cajas para guardar datos

Una variable es un nombre que apunta a un valor almacenado en memoria. En Python **no necesitas declarar el tipo** explícitamente; el intérprete lo deduce solo. Esto se llama *tipado dinámico*.

```
nombre = "Ada Lovelace" # str → cadena de texto
edad   = 28             # int  → número entero
altura = 1.72           # float → decimal
activo = True           # bool → verdadero/falso
nada   = None           # NoneType → ausencia de valor
```

 Las variables en Python distinguen mayúsculas de minúsculas: `edad` y `Edad` son variables diferentes.

Tipos de datos fundamentales

int

Números enteros sin decimales.

```
x = 42
```

```
año = 2025
```

float

Números con parte decimal.

```
pi = 3.1416
```

```
tasa = 0.05
```

str

Cadenas de texto entre comillas.

```
"Hola"
```

```
'Python'
```

bool

Verdadero o falso.

```
True
```

```
False
```

NoneType

Representa la ausencia de valor.

```
resultado = None
```

Puedes verificar el tipo de cualquier variable con la función incorporada `type(variable)`.

Operadores en Python

Los operadores son los símbolos que permiten realizar cálculos, comparaciones y operaciones lógicas. Conocerlos bien es imprescindible para escribir cualquier algoritmo.

Operadores aritméticos

- + suma
- resta
- * multiplicación
- / división (float)
- // división entera
- % módulo (resto)
- ** potencia

Operadores de comparación

- == igual
- != distinto
- > mayor que
- < menor que
- >= mayor o igual
- <= menor o igual

Operadores lógicos

- and y lógico
- or o lógico
- not negación

Operadores en acción

Aritméticos

```
resultado = 10 ** 2    # 100
```

```
resto    = 17 % 5      # 2
```

Comparación → devuelven True o False

```
print(10 > 3)          # True
```

```
print("hola" == "HOLA") # False
```

Lógicos

```
edad = 20
```

```
tiene_dni = True
```

```
puede_votar = edad >= 18 and tiene_dni
```

```
print(puede_votar)    # True
```

Fíjate en que `==` compara valores, mientras que `=` asigna un valor a una variable. ¡Es uno de los errores más frecuentes entre principiantes!



CONTROL DE FLUJO

Condicionales: tomando decisiones

Los condicionales permiten que tu programa ejecute diferentes bloques de código según se cumplan o no ciertas condiciones. En Python se escriben con `if`, `elif` y `else`, y la **indentación** (sangría) define los bloques — no se usan llaves como en Java.

Estructura de los condicionales

Sintaxis

```
if condicion_1:  
    # bloque 1  
elif condicion_2:  
    # bloque 2  
else:  
    # bloque por defecto
```

Ejemplo práctico

```
nota = 7.5  
  
if nota >= 9:  
    print("Sobresaliente")  
elif nota >= 7:  
    print("Notable")  
elif nota >= 5:  
    print("Aprobado")  
else:  
    print("Suspenso")
```

La condición `nota >= 7` se evalúa solo si la anterior (`nota >= 9`) fue `False`.

Bucles: repitiendo acciones

Los bucles ejecutan un bloque de código múltiples veces. Python ofrece dos tipos principales:

Bucle `for`

Itera sobre una secuencia (lista, rango, cadena...). Sabes de antemano cuántas veces se ejecutará.

```
for i in range(5):  
    print(i) # 0 1 2 3 4
```

```
frutas = ["manzana", "pera", "uva"]  
for fruta in frutas:  
    print(fruta)
```

Bucle `while`

Repite mientras una condición sea verdadera. Útil cuando no sabes cuántas iteraciones necesitarás.

```
contador = 0  
while contador < 3:  
    print(contador)  
    contador += 1  
# Resultado: 0 1 2
```

⚠ Un bucle `while` sin condición de salida bien definida puede ejecutarse infinitamente y bloquear tu programa.

Comunicándose con el usuario

Todo programa útil necesita recibir datos del exterior y mostrar resultados. Python lo hace de manera muy sencilla con dos funciones integradas: `print()` e `input()`.

Salida: `print()`

```
# Texto simple
print("Hola, mundo")

# Variables
nombre = "Lucía"
print("Hola,", nombre)

# f-strings (recomendado)
edad = 21
print(f"{nombre} tiene {edad} años")
```

Entrada: `input()`

```
# Siempre devuelve str
nombre = input("¿Cómo te llamas? ")

# Convierte a número si lo necesitas
edad = int(input("¿Cuántos años tienes? "))

print(f"¡Hola {nombre}!")
print(f"En 10 años tendrás {edad+10}")
```

Importante: `input()` siempre devuelve texto. Usa `int()` o `float()` para convertirlo.

Ejemplo completo: entrada, proceso y salida

```
# Conversor de temperatura: Celsius → Fahrenheit
celsius_str = input("Introduce la temperatura en Celsius: ")
celsius = float(celsius_str)

fahrenheit = celsius * 9 / 5 + 32

if fahrenheit >= 100:
    aviso = "
```

Comparativa: Python vs Java

Si has programado en Java, esta tabla te ayudará a traducir mentalmente los conceptos que ya conoces a la sintaxis de Python.

Aspecto	Java	Python
Declaración de variable	<code>int x = 5;</code>	<code>x = 5</code>
Tipado	Estático y explícito	Dinámico e inferido
Punto y coma	Obligatorio al final de cada línea	No se usa
Bloques de código	Llaves <code>{ }</code>	Indentación (espacios)
Condicional	<code>if (x > 0) { ... }</code>	<code>if x > 0:</code>
Bucle for	<code>for (int i=0; i<5; i++) { }</code>	<code>for i in range(5):</code>
Imprimir en pantalla	<code>System.out.println("Hi");</code>	<code>print("Hi")</code>
Leer entrada	<code>Scanner sc = new Scanner(System.in);</code>	<code>input("Mensaje: ")</code>
Comentario	<code>// comentario</code>	<code># comentario</code>

La diferencia más importante: la indentación

☕ Java — llaves para delimitar bloques

```
if (nota >= 5) {  
    System.out.println("Aprobado");  
} else {  
    System.out.println("Suspenso");  
}
```

Las llaves definen dónde empieza y acaba cada bloque. La indentación es una convención de estilo, no una regla del lenguaje.

🐍 Python — indentación obligatoria

```
if nota >= 5:  
    print("Aprobado")  
else:  
    print("Suspenso")
```

La indentación **es parte de la sintaxis**. Un error de sangría provoca un `IndentationError`. Usa siempre 4 espacios por nivel.

Hora de practicar

La mejor manera de afianzar estos conceptos es escribir código. Aquí tienes tres ejercicios progresivos para trabajar en tu entorno local o en Jupyter:

1 Calculadora de IMC

Pide al usuario su peso (kg) y altura (m). Calcula el Índice de Masa Corporal (`imc = peso / altura**2`) y muestra si está por debajo del peso, en el rango normal, con sobrepeso o en obesidad usando condicionales.

2 Tabla de multiplicar

Pide un número al usuario y usa un bucle `for` con `range(1, 11)` para mostrar su tabla de multiplicar completa con formato: `7 x 3 = 21`.

3 Adivina el número

Genera un número aleatorio entre 1 y 20 con `import random; numero = random.randint(1, 20)`. Usa un bucle `while` para pedir intentos al usuario y dar pistas («muy alto», «muy bajo») hasta acertar.

Resumen de la sesión

Has recorrido los fundamentos de Python con ojo puesto en la IA. Esto es lo que te llevas hoy:



Python + IA

Legible, potente y con el mayor ecosistema de bibliotecas para IA y datos.



Entorno

venv + pip + Jupyter + VS Code + Git: tu caja de herramientas profesional.



Variables y tipos

int, float, str, bool y None. Tipado dinámico sin declaraciones explícitas.



Control de flujo

if/elif/else para decidir; for y while para repetir. La indentación es ley.



E/S

print() e input() para dialogar con el usuario. Convierte con int() o float().



Próxima sesión: Estructuras de datos en Python — listas, tuplas, diccionarios y conjuntos. ¡No olvides completar los ejercicios antes!